**What are let, var, and const? (IMP)**

In JavaScript, variables are declared using three keywords:

let nr1 = 12;

var nr2 = 8;

const PI = 3.14159;

Each of these keywords defines a variable differently in terms of **scope**, **hoisting**, and **reassignment**.

**1. var**

**Characteristics:**

- Introduced in **ES5 and earlier**
- **Function-scoped**: Accessible within the function it is declared in.
- **Hoisted**: Declared at the top of the scope, but **not initialized**.
- **Can be re-declared and reassigned**.

**Example:**

```
function testVar() {
  var x = 10;
  if (true) {
    var x = 20; // same variable!
    console.log(x); // 20
  }
  console.log(x); // 20
}
```

**Use with Caution:**

- It can **lead to bugs** because of its global/function-level scope.
- Avoid using var in modern JavaScript.

**2. let**

**Characteristics:**

- Introduced in **ES6 (2015)**
- **Block-scoped**: Only accessible within { } block where it was declared.
- **Hoisted**: But **not initialized** — using it before declaration causes error.
- **Can be reassigned**, but **not re-declared in the same scope**.

**Example:**

```
function testLet() {
  let y = 10;
  if (true) {
    let y = 20; // different variable (block-scoped)
    console.log(y); // 20
  }
  console.log(y); // 10
}
```

**Best practice:**

- Use let when the variable **needs to change later**.
- Safer and more predictable than var.

### 3. const

**Characteristics:**

- Also introduced in **ES6**
- **Block-scoped** like let.
- **Hoisted but not initialized**.
- **Cannot be reassigned** after declaration.
- **Must be initialized** at the time of declaration.

**Example:**

```
const PI = 3.14159;

PI = 3; //  Error: Assignment to constant variable.
```

**However**, if the const variable holds an object or array, its **properties/elements** can still be changed:

```
const arr = [1, 2, 3];

arr.push(4); //  Allowed

arr = [4, 5]; //  Error
```

**Best practice:**

- Use const by default.
- Only use let when you **expect** the value to change.

**Comparison Table**

| Feature | var | let | const |
|---|---|---|---|
| Scope | Function | Block | Block |
| Hoisting | Yes | Yes | Yes |
| Initialization | Undefined | No | No |
| Reassignment | Yes | Yes | No |
| Redeclaration | Yes | No | No |
| Best Use Case | Avoid | Variable that may change | Fixed values |



**Summary (Key Points)**

- var → function-scoped, can cause bugs due to redeclaration and hoisting.
- let → block-scoped, preferred for mutable variables.
- const → block-scoped, preferred for constants and immutable references.

## JavaScript Primitive Data Types

JavaScript is a **loosely typed** language, which means variable types are determined by their **assigned values**, not their declarations.

For example:

```
let a = 5;       // JavaScript automatically assigns it as type 'number'
let name = "Avijit";  // Automatically 'string'
```

## What Are Primitive Data Types?

Primitive data types are the **most basic units** of data in JavaScript. They are immutable (cannot be changed once created) and store **single, simple values**.



## JavaScript has 7 primitive types:

| Type | Description |
|---|---|
| String | Text or sequence of characters |
| Number | Integer, float, hexadecimal, exponential… |
| BigInt | Very large integers |
| Boolean | Logical true or false |
| Symbol | Unique, immutable value |
| undefined | Value not assigned |
| null | Empty or unknown value |

## 1. String

- Used to store **text values**
- Defined using:
  - 'single quotes'
  - "double quotes"
  - `template literals` (backticks allow variables inside)

**Example:**

```
let str1 = 'Hello!';
let str2 = "Hi!";
let name = "Avijit";
let msg = `Welcome, ${name}`;  // Template literal
```

**Beware of quote confusion:**

```
let msg = 'Let's learn JS'; //  Error
let msg = "Let's learn JS"; //  OK
```

**Escape Characters:**

Use \ to escape quotes or special characters:

```
let str = "He said, \"Hello!\"";
let path = "C:\\Users\\Avijit";
let multiLine = "Line1\nLine2";
```

## 2. Number

- Stores all numeric values: integers, decimals, exponential, hex, binary, etc.
- Internally uses **64-bit floating point format**

**Examples:**

let int = 10;

let float = 12.5;

let exp = 1.5e4; // 15000

let hex = 0xFF;  // 255

let bin = 0b101; // 5

let oct = 0o10;  // 8

**3. BigInt**

- Used for **very large integers** beyond the Number type limit.
- Postfixed with n

**Example:**

let big = 9007199254740991n;

**Cannot mix BigInt with Number:**

let result = big + 10; // ❌ Error!

**4. Boolean**

- Represents either **true** or **false**

**Example:**

let isActive = true;

let isDeleted = false;

**5. Symbol (ES6)**

- Always **unique**, even if they have the same description
- Often used for **object properties** to avoid name collisions

**Example:**

```
let sym1 = Symbol("id");

let sym2 = Symbol("id");


console.log(sym1 === sym2); // false
```



**6. undefined**

- A variable declared but **not assigned** gets value undefined by default.

**Example:**

```
let x;

console.log(x); // undefined
```

**Avoid assigning undefined manually:**

```
let badPractice = undefined; // Don't do this!
```

**7. null**

- A **deliberate assignment** to show that the variable is **empty or unknown**

**Example:**

let user = null;

**Gotcha:**

typeof null; // Returns "object" (a known JavaScript bug)

**typeof Operator**

Use typeof to check the type of a variable:



```
console.log(typeof "Hello");   // string
console.log(typeof 10);        // number
console.log(typeof true);      // boolean
console.log(typeof undefined); // undefined
console.log(typeof null);      // object ❗
console.log(typeof Symbol("a")); // symbol
```

**Type Conversion**

JavaScript often **converts types automatically**, but manual conversion is more reliable.

**Type Conversion Methods:**

| Function | Converts To |
|---|---|
| String() | String |
| Number() | Number |
| Boolean() | Boolean |

**Examples:**

```
let num = 5;
console.log(String(num));    // "5"

let str = "123";
console.log(Number(str));    // 123

let isLogged = "";
console.log(Boolean(isLogged)); // false
```

**Special Cases:**

```
Number(null);      // 0
Number("");        // 0
Number("hello");   // NaN

Boolean("false");  // true
Boolean("");       // false
```

```
Boolean(0);       // false
Boolean("0");     // true
```

**Practice Exercise Output**

```
let str1 = 'Laurence';
let str2 = "Svekis";
let val1 = undefined;
let val2 = null;
let myNum = 1000;

console.log(typeof str1); // string
console.log(typeof str2); // string
console.log(typeof val1); // undefined
console.log(typeof val2); // object
console.log(typeof myNum); // number
```

**Summary**

| Type | Example | typeof Result |
|---|---|---|
| String | "Hello" | "string" |
| Number | 100, 12.5 | "number" |
| BigInt | 1234567890123456 78n | "bigint" |
| Boolean | true, false | "boolean" |
| Symbol | Symbol("id") | "symbol" |
| undefined | let a; | "undefined" |
| null | let b = null; | "object" ❗ |

## JavaScript Operators

**Operators** are used to **perform operations on variables and values**. They're essential for doing math, modifying values, and performing logic in JavaScript.

### Arithmetic Operators

Used to perform **mathematical operations**.

| Operator | Description | Example | Output |
|---|---|---|---|
| + | Addition | 10 + 5 | 15 |
| - | Subtraction | 10 - 5 | 5 |
| * | Multiplication | 10 * 5 | 50 |
| / | Division | 10 / 2 | 5 |
| % | Modulus (remainder) | 10 % 3 | 1 |
| ** | Exponentiation | 2 ** 3 | 8 |



### Addition (+)

```
let a = 12;
let b = 14;
let result = a + b;
console.log(result); // 26
```

Also used for **string concatenation**:

```
let greeting = "Hello ";
let subject = "World";
console.log(greeting + subject); // Hello World
```

**Subtraction (-)**

```
let result = 20 - 4; // 16
let errorResult = "Hi" - 3;
console.log(errorResult); // NaN
```

Subtracting a number from a non-number results in **NaN**.

**Multiplication (*)**

```
console.log(15 * 10);    // 150
console.log("Hi" * 3);   // NaN
```

**Division (/)**

```
console.log(30 / 5);     // 6
```

**Exponentiation (**)**

```
console.log(2 ** 3);     // 8
console.log(9 ** 0.5);   // 3 (Square root)
```

**Modulus (%)**

Finds the **remainder** after division.

```
console.log(10 % 3);  // 1
console.log(8 % 2);   // 0
console.log(15 % 4);  // 3
```

Helpful for:

- Checking if a number is even (num % 2 === 0)
- Working with time (125 % 60 gives leftover minutes)

**Unary Operators (Increment & Decrement)**

Unary operators need **only one operand**.

| Operator | Description | Example | Result |
|---|---|---|---|
| ++ | Increment by 1 | x++ | Adds 1 |
| -- | Decrement by 1 | x-- | Minus 1 |

**Example:**

```
let x = 4;
x++; // 5
x--; // 4
```

**Prefix vs Postfix**

**Postfix (x++, x--):**

Returns the value **before** incrementing/decrementing.

```
let a = 2;
console.log(a++); // 2
console.log(a);   // 3
```

**Prefix (++x, --x):**

Returns the value **after** incrementing/decrementing.

```
let b = 2;
console.log(++b); // 3
```

**Combined Example:**

```
let nr1 = 4;
let nr2 = 5;
let nr3 = 2;

console.log(nr1++ + ++nr2 * nr3++);
//       4    +   6   *   2
//       4 + 12 = 16
```

**Operator Precedence**

The order in which operations are **evaluated** in an expression.

**Precedence Table:**

| Precedence | Operator Type | Symbols | Example |
|---|---|---|---|
| 1 | Grouping | () | (a + b) |
| 2 | Exponentiation | ** | a ** b |
| 3 | Prefix Increment/Decrement | ++, -- | ++a, --a |
| 4 | Multiplication/Division/Modulus | * / % | a * b / c |
| 5 | Addition/Subtraction | + - | a + b - c |



**Example:**

```
let result = (2 + 3) * 4 ** 2;
// 2 + 3 = 5
// 4 ** 2 = 16
// 5 * 16 = 80
```